

More on Types

So far, we've seen how to *work* with types, and we want to give a somewhat more robust account of their theory.

The hierarchy

We've seen that propositions are all types of a certain kind **Prop**; and that $\mathbb{N}$ or $\mathbb{R}$ are types of a different kind (indeed, both have more than one term!), called **Type**.

There is actually a whole hierarchy of kinds of types

```
Prop : Type 0 : Type 1 : ... : Type n : ...
```

So, **Prop** is a *term* of the type **Type 0**, itself a *term* of the type **Type 1**, etc.

Lean shortens **Type 0** to **Type**, omitting the index. It is where most known mathematical objects (like $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{C}$, etc) live. **Sort *** is either **Type (*+1)** or **Prop** in the sense that **Sort 0 = Prop**, **Sort 1 = Type 0**, **Sort 37 = Type 36**...

⌘

Dependent types

The type theory that Lean is built upon axiomatises the existence of certain constructions that allow to construct new types out of known ones.

+++ Function types

Given two types **X** and **Y**, it exists the type **X → Y**. Its terms are written

```
λ (x : X) ↦ (f x) : Y
```

or

```
fun (x : X) ↦ f x
```

These terms can be interpreted as functions from **X** to **Y**, in the sense that if $x_0 : X$ and $f : X \rightarrow Y$ then $f x$ is a term in **Y**.

⌘

+++

+++ Π -types and Σ -types

Π -types

What is the type of

```
fun (α : Type) → (fun x : α → x)
```

namely the assignment sending a type to its identity function?

The "problem" is that for every element in the domain, the image lies in a different place: there is no "codomain".

It belongs to the Π -type (called pi-type, or forall-type, or *dependent product*)

```
Π (α : Type), α → α
∀ (α : Type), α → α
(α : Type) → (α → α)
```

More generally, given a type A (where $A = \text{Sort } u$ is allowed), seen as an indexing set/type, and a function $I : A \rightarrow \text{Sort } v$, seen as an "indexing family",

```
Π (a : A), I a
∀ (a : A), I a
(a : A) → I a
```

is the type whose terms are functions assigning to each $a : A$ a term $x_a : I a$. When A is non-empty, they are collections (a, x_a) for a spanning all terms of type A and where $x_a : I a$. Its terms are written $\lambda a : A \mapsto x_a$, or $\text{fun } a : A \mapsto x_a$.

- If you've got a geometric intuition, this looks very much like a fibration, where A is the base and $I a$ is the fiber above $a : A$.
- As the λ or fun notation suggest, $X \rightarrow Y$ is a special case of a Π -type, where $I : X \rightarrow \text{Sort } v$ is the constant function $\text{fun } x \mapsto I x = Y$.
- From the hierarchy point of view, if $A : \text{Sort } u$ and $I : A \rightarrow \text{Sort } v$, then $(a : A) \rightarrow I a$ lives in $\text{Sort } (\max u v)$ except when $v = 0$ in which case $(a : A) \rightarrow I a : \text{Prop}$. This last point is the "impredicativity" of the underlying type-theory.

Σ -types

Similarly, terms of the Σ -type

```
Σ (a : A), I a
(a : A) × I a
```

are pairs $\langle a, x_a \rangle$ where $x_a : I a$ (for technical reasons, we need here that $A : \text{Type } u$ and that $I : A \rightarrow \text{Type } v$: if you really want to use Sort use Σ' , or $\times'$). Type $\times$ (*resp.* $\times'$) as $\backslash x$ (*resp.* $\backslash x'$).

- These constructions of types that depend on terms give the name "dependent type theory" (or "dependent λ -calculus") to the underlying theory.
- From the hierarchy point of view, if $A : \text{Type } u$ and $I : A \rightarrow \text{Type } v$, then $(a : A) \times I\ a$, is in $\text{Sort } (\max\ u\ v)$.

If $A : \text{Sort } u$ and $I : A \rightarrow \text{Sort } v$, then $(a : A) \times' I\ a : \text{Sort } (\max\ (\max\ 1\ u)\ v)$.

⌘

+++

+++ $\forall$ and $\exists$

Universal quantifier

Consider the type

$$\forall n \in \mathbb{N}, \exists p \text{ prime such that } n < p$$

It is a Π -type, with $I : \mathbb{N} \rightarrow \text{Prop}$ being $I := \text{fun } n \mapsto \exists p \text{ prime, } n < p$.

Euclid's proof is a *term* of the above type.

- You can *prove* a $\forall$ by **introducing** a variable via **intro** x , and by proving $P\ x$.
- If you have $H : \forall x : \alpha, P\ x$ and also a term $y : \alpha$, you can specialise H to y :

```
specialize H y (:= P y)
```

- If the goal is $\vdash P\ y$, you might simply want to do **exact** $H\ y$, remembering that implications, $\forall$ and functions are all the same thing.

Existential quantifier

A statement

$$\exists n \in \mathbb{N}, n^2 + 37 \cdot n < 2^n$$

lives in **Prop**, so you *cannot extract a witness from an existential proof*, unless you want to extract a term in some other $P : \text{Prop}$.

But,

- To prove $\exists x, P\ x$, you first produce x , and then prove it satisfies $P\ x$: once you have constructed x , do **use** x to have Lean ask you for $\vdash P\ x$.
- If you have $H : \exists x, P\ x$, do **obtain** $\langle x, hx \rangle := H$ to obtain the term x together with a proof that $P\ x$.

⌘

+++

+++ $\neg$ and Proofs by contradiction

- Given $P : \text{Prop}$, the *definition* of $\neg P$ is

$P \rightarrow \text{False}$

- The `exfalso` tactic changes *any* goal to proving `False` (useful if you have an *assumption* $\dots \rightarrow \text{False}$).
- Proofs by contradiction, introduced using the `by_contra` tactic, require you to prove `False` assuming `not (the goal)`: if your goal is $\vdash P$, typing `by_contra h` creates

```
h : ¬ P
⊢ False
```

The difference between `exfalso` and `by_contra` is that the first does not introduce anything, and forgets the actual goal; the second negates the goal and asks for `False`.

- `contrapose` is the tactic that changes a goal $P \rightarrow Q$ to $\neg Q \rightarrow \neg P$.

⌘

+++

Inductive Types

So far, we

- met some abstract types $\alpha, \beta, T : \text{Type}$, and variations like $\alpha \rightarrow T$ or $\beta \rightarrow \text{Type}$;
- also met a lot of types $P, Q, (1 = 2) \wedge (0 \leq 5) : \text{Prop}$;
- struggled a bit with $h : (2 = 3)$ *versus* $(2 = 3) : \text{Prop}$;
- also met $\mathbb{N}, \mathbb{R}$...

How can we *construct* new types, or how have these been constructed? For instance, $\mathbb{R}$, or `True : Prop` or the set of even numbers? Using **inductive types**.

+++ Perspectives

- Theoretical*: this is (fun & interesting, but) beyond the scope of this course: it is very much discussed in the references.
- Practical*: think of $\mathbb{N}$ and surf the wave. It has two **constructors**: the constant $0 : \mathbb{N}$ and the function `succ` : $\mathbb{N} \rightarrow \mathbb{N}$, and every $n : \mathbb{N}$ is of either form. Moreover, it satisfies **induction**/recursion.

+++

For example

```
inductive NiceType
| Tom : NiceType
| Jerry : NiceType
```

```
| f : NiceType → NiceType
| g : ℕ → NiceType → NiceType → NiceType
```

constructs the "minimal/smallest" type `NiceType` whose terms are

1. Either `Tom`;
2. Or `Jerry`;
3. Or an application of `f` to some previously-defined term;
4. Or an application of `g` to a natural and a pair of previously-defined terms.

For example, `f (g 37 Tom Tom) : NiceType`.

- Every inductive type comes with its *recursor*, that is automatically constructed by Lean: it builds dependent functions *out of the inductive type being constructed* by declaring the value that should be assigned to every constructor.
- In order to
 1. construct terms of type `NiceType` you can use the ... *constructors*!;
 2. access terms of type `NiceType` (in a proof, say), use the tactic `cases` (or `rcases`): the proofs for `Tom` and for `Jerry` might differ, so a case-splitting is natural.

⌘

Every type in Lean is either a function type, a quotient type or an inductive type

By *every* we mean `True`, `False`, `Bool`, `P ∧ Q`... every! And among those,

If and only if statements:

`↔ : Prop → Prop → Prop`, giving rise to `P ↔ Q`: it is an inductive type (of course), with

- two parameters (`P` and `Q`)
- one constructor, itself made of
 - two fields: `Iff.mp : P → Q` and `Iff.mpr : Q → P`

Calling `#print Iff` produces:

```
structure Iff (P Q : Prop) : Prop
  number of parameters: 2
  fields:
    Iff.mp : P → Q
    Iff.mpr : Q → P
  constructor: Iff.intro {P Q : Prop} (mp : P → Q) (mpr : Q → P) : P ↔ Q
```

An equivalence can be either *proved* or *used*. This amounts to saying that:

- A goal `⊢ P ↔ Q` can be broken into the goals `⊢ P → Q` and `⊢ Q → P` using `constructor`: indeed, to prove `⊢ P ↔ Q` amounts to creating *the unique term* of `P ↔ Q` which has two constructors;

- The projections $(P \leftrightarrow Q).mp$ (or $(P \leftrightarrow Q).1$) and $(P \leftrightarrow Q).mpr$ (or $(P \leftrightarrow Q).2$) are the implications $P \rightarrow Q$ and $Q \rightarrow P$, respectively. These are the two "components" of the term in $P \leftrightarrow Q$.

